

React with Frameworks, State Management, Rendering Modes, Build Tools, and Frontend Architecture

This file is intentionally written as a reference, not a checklist. The goal is to answer three things for every major topic:

1. What is it?
2. When should you use it?
3. What does a realistic setup or usage pattern look like in code?

This is the level of detail you want for tougher L4/L5-style frontend and full-stack interviews, where naming a tool is not enough. You need to show that you know how to wire it, use it, and justify it.

1) First Principle: Not All State Is the Same

A lot of bad React architecture comes from using one tool for every kind of state.

Separate state into these buckets first:

- Local UI state: modal open/close, tab selection, hover, input draft.
- Shared client state: theme, auth summary, wizard progress, cart UI state.
- Server state: API data, loading, retries, invalidation, staleness.
- URL state: filters, pagination, sort order, route params.
- Form state: dirty fields, touched fields, validation errors, submission state.
- Derived state: totals, filtered lists, booleans computed from other values.

1.1 Local State Example

```
function ProductFilters() {
  const [isOpen, setIsOpen] = useState(false);
  const [query, setQuery] = useState("");

  return (
    <section>
      <button onClick={() => setIsOpen((open) => !open)}>
        {isOpen ? "Hide" : "Show"} Filters
      </button>
      {isOpen && (
        <input
          value={query}
          onChange={(event) => setQuery(event.target.value)}
          placeholder="Search products"
        />
      )}
    </section>
  );
}
```

Use local state when:

- the state belongs to one small subtree,
- you do not need persistence in the URL,

- you do not need global access,
- the state is not server-backed cache data.

1.2 URL State Example

```
import { useSearchParams } from "react-router-dom";

function ProductListPage() {
  const [searchParams, setSearchParams] = useSearchParams();
  const sort = searchParams.get("sort") ?? "price";

  return (
    <select
      value={sort}
      onChange={(event) => {
        setSearchParams({ sort: event.target.value });
      }}
    >
      <option value="price">Price</option>
      <option value="rating">Rating</option>
    </select>
  );
}
```

Use URL state when:

- the state should survive refreshes,
- users should be able to bookmark/share it,
- analytics or SEO depend on the state,
- route navigation should reflect it.

1.3 Derived State Example

```
function CartSummary({ items }) {
  const total = items.reduce((sum, item) => sum + item.price * item.quantity, 0);
  const itemCount = items.reduce((sum, item) => sum + item.quantity, 0);

  return <p>{itemCount} items, total ${total}</p>;
}
```

Interview line:

- If a value can be calculated from existing state and props during render, prefer deriving it instead of storing it separately.

2) Plain React Patterns Before External Libraries

Before adding Redux, Query libraries, or atom stores, make sure plain React is not enough.

2.1 Lifting State Up

```
function SearchPage() {
  const [query, setQuery] = useState("");

  return (
    <>
      <SearchInput query={query} setQuery={setQuery} />
      <SearchResults query={query} />
    </>
  );
}

function SearchInput({ query, setQuery }) {
  return <input value={query} onChange={(event) => setQuery(event.target.value)} />;
}

function SearchResults({ query }) {
  return <div>Showing results for: {query}</div>;
}
```

Use this when:

- two or three sibling components need the same state,
- the state still has a single clear owner,
- global state would be overkill.

2.2 Context for Low-Frequency Shared State

```
const ThemeContext = createContext(null);

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  const value = useMemo(() => ({ theme, setTheme }), [theme]);
  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
}

function ThemeToggle() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      Current theme: {theme}
    </button>
  );
}
```

Context is best for:

- theme,
- locale,
- auth summary,

- feature flags,
- stable shared services.

Context is not ideal for:

- high-frequency app-wide writes,
- complex async data flows,
- normalized entity collections,
- server-state caching.

2.3 `useReducer` for Event-Driven Local State

```
const initialState = {
  step: 1,
  shippingAddress: null,
  paymentMethod: null,
  error: null
};

function checkoutReducer(state, action) {
  switch (action.type) {
    case "shippingSaved":
      return { ...state, shippingAddress: action.payload, step: 2 };
    case "paymentSaved":
      return { ...state, paymentMethod: action.payload, step: 3 };
    case "failed":
      return { ...state, error: action.payload };
    default:
      return state;
  }
}

function CheckoutFlow() {
  const [state, dispatch] = useReducer(checkoutReducer, initialState);

  return (
    <div>
      <p>Current step: {state.step}</p>
      <button
        onClick={() =>
          dispatch({ type: "shippingSaved", payload: { city: "Bengaluru" } })
        }
      >
        Save shipping
      </button>
    </div>
  );
}
```

Use `useReducer` when:

- transitions are easier to describe as events,

- many fields change together,
- you want a testable reducer without introducing global state.

3) Redux Core

Redux is predictable global state management built around:

- a single store,
- actions that describe what happened,
- reducers that calculate the next state,
- subscriptions/selectors that read state.

3.1 Classic Redux Mental Model

```
const initialState = { count: 0 };

function counterReducer(state = initialState, action) {
  switch (action.type) {
    case "counter/incremented":
      return { ...state, count: state.count + 1 };
    default:
      return state;
  }
}
```

3.2 When Redux Is Actually a Good Choice

Use Redux when:

- state is shared across large parts of the app,
- transitions are business-event heavy,
- debugging and traceability matter,
- middleware and extensibility matter,
- you want predictable state transitions across teams.

Examples:

- admin dashboards,
- workflow builders,
- collaboration tools,
- apps with websockets + optimistic updates + undo/redo.

3.3 When Redux Is the Wrong Tool

Avoid Redux when:

- most of your complexity is API caching,
- state is mostly local,
- you are building a content site,
- you have not first tried context + reducer + URL state.

Interview line:

- Redux is strongest for shared client-side application state, not for replacing every other React pattern.

4) Redux Toolkit: The Recommended Redux Setup

Redux Toolkit is the official modern way to write Redux.

Main pieces you should know:

- `configureStore`
- `createSlice`
- `createAsyncThunk`
- `createEntityAdapter`
- `createSelector`
- RTK Query

4.1 Project Setup

Typical install:

```
npm install @reduxjs/toolkit react-redux
```

4.2 Main Store Setup

```
// src/app/store.js
import { configureStore } from "@reduxjs/toolkit";
import authReducer from "../features/auth/authSlice";
import cartReducer from "../features/cart/cartSlice";

export const store = configureStore({
  reducer: {
    auth: authReducer,
    cart: cartReducer
  }
});
```

What `configureStore` includes by default:

- Redux DevTools integration,
- thunk middleware,
- development-time checks for accidental mutations,
- development-time checks for non-serializable values.

4.3 Wiring the Store to React

```
// src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import { store } from "../app/store";
import App from "../App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
```

```

    <App />
  </Provider>
);

```

Without `<Provider>`, React components cannot use `useSelector` or `useDispatch`.

4.4 Creating a Slice

```

// src/features/cart/cartSlice.js
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  items: [],
  coupon: null
};

const cartSlice = createSlice({
  name: "cart",
  initialState,
  reducers: {
    itemAdded(state, action) {
      const existingItem = state.items.find((item) => item.id === action.payload.id);
      if (existingItem) {
        existingItem.quantity += 1;
      } else {
        state.items.push({ ...action.payload, quantity: 1 });
      }
    },
    itemRemoved(state, action) {
      state.items = state.items.filter((item) => item.id !== action.payload);
    },
    couponApplied(state, action) {
      state.coupon = action.payload;
    }
  }
});

export const { itemAdded, itemRemoved, couponApplied } = cartSlice.actions;
export default cartSlice.reducer;

```

Interview point:

- RTK uses Immer, so this reducer code looks mutative, but it produces immutable updates safely.

4.5 Dispatching and Selecting State in Components

```

import { useDispatch, useSelector } from "react-redux";
import { itemAdded, itemRemoved } from "../cartSlice";

function CartPanel() {
  const dispatch = useDispatch();

```

```

const items = useSelector((state) => state.cart.items);

return (
  <div>
    <button
      onClick={() => dispatch(itemAdded({ id: "shoe-1", name: "Running Shoe", price: 80
    })))}
    >
      Add shoe
    </button>

    <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} x {item.quantity}
          <button onClick={() => dispatch(itemRemoved(item.id))}>Remove</button>
        </li>
      ))}
    </ul>
  </div>
);
}

```

4.6 Typed Hooks Pattern

In TypeScript apps, teams usually create reusable hooks.

```

// src/app/hooks.ts
import { useDispatch, useSelector } from "react-redux";
import { store } from "../store";

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();

```

4.7 Selectors

Selectors centralize read logic and reduce duplication.

```

// src/features/cart/cartSelectors.js
export const selectCartItems = (state) => state.cart.items;
export const selectCartCount = (state) =>
  state.cart.items.reduce((sum, item) => sum + item.quantity, 0);
export const selectCartTotal = (state) =>
  state.cart.items.reduce((sum, item) => sum + item.price * item.quantity, 0);

```

Use selectors because:

- state shape changes are easier to isolate,

- derived logic becomes reusable,
- components stay simpler.

5) What Is Thunk and When Do You Use It?

A thunk is a function returned from another function. In Redux middleware terms, a thunk lets you dispatch a function instead of a plain action object.

That function receives:

- `dispatch`
- `getState`

5.1 Why Thunks Exist

You need them when an action cannot be completed synchronously with a single reducer transition.

Examples:

- fetch data,
- submit a form,
- check state before dispatching,
- call multiple actions in sequence.

5.2 Manual Thunk Example

```
// src/features/auth/authThunks.js
export function loginUser(credentials) {
  return async function loginThunk(dispatch, getState) {
    dispatch({ type: "auth/loginStarted" });

    try {
      const response = await fetch("/api/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(credentials)
      });

      if (!response.ok) {
        throw new Error("Login failed");
      }

      const data = await response.json();
      dispatch({ type: "auth/loginSucceeded", payload: data.user });
    } catch (error) {
      dispatch({ type: "auth/loginFailed", payload: error.message });
    }
  };
}
```

5.3 Dispatching a Thunk

```
function LoginButton() {
  const dispatch = useDispatch();

  return (
    <button
      onClick={() =>
        dispatch(loginUser({ email: "alice@example.com", password: "secret" }))
      }
    >
      Login
    </button>
  );
}
```

5.4 createAsyncThunk

Redux Toolkit provides a standard thunk helper.

```
// src/features/users/usersThunks.js
import { createAsyncThunk } from "@reduxjs/toolkit";

export const fetchUsers = createAsyncThunk("users/fetchUsers", async () => {
  const response = await fetch("/api/users");
  if (!response.ok) {
    throw new Error("Could not fetch users");
  }
  return response.json();
});
```

5.5 Handling Async States in a Slice

```
// src/features/users/usersSlice.js
import { createSlice } from "@reduxjs/toolkit";
import { fetchUsers } from "../usersThunks";

const usersSlice = createSlice({
  name: "users",
  initialState: {
    items: [],
    status: "idle",
    error: null
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.status = "loading";
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {

```

```

        state.status = "succeeded";
        state.items = action.payload;
    })
    .addCase(fetchUsers.rejected, (state, action) => {
        state.status = "failed";
        state.error = action.error.message;
    });
}
});

export default usersSlice.reducer;

```

Use `createAsyncThunk` when:

- the async process is still part of client-app workflow state,
- you want standard pending/fulfilled/rejected action lifecycle,
- you are not solving generic cache/invalidation problems at scale.

Do not force it when the real need is server-state caching. That is where RTK Query or TanStack Query often fit better.

6) `createEntityAdapter`

This is one of the most underrated RTK utilities.

Use it when you manage entity collections by ID.

```

import { createEntityAdapter, createSlice } from "@reduxjs/toolkit";

const usersAdapter = createEntityAdapter({
  sortComparer: (a, b) => a.name.localeCompare(b.name)
});

const usersSlice = createSlice({
  name: "users",
  initialState: usersAdapter.getInitialState({ status: "idle" }),
  reducers: {
    usersReceived(state, action) {
      usersAdapter.setAll(state, action.payload);
    },
    userUpdated(state, action) {
      usersAdapter.updateOne(state, action.payload);
    }
  }
});

export const { usersReceived, userUpdated } = usersSlice.actions;
export default usersSlice.reducer;

```

Generated selectors:

```

export const {
  selectAll: selectAllUsers,

```

```
  selectById: selectUserById,  
  selectIds: selectUserIds  
} = usersAdapter.getSelectors((state) => state.users);
```

Benefits:

- normalized state shape,
- easier updates,
- reusable selectors,
- better performance for large collections.

7) RTK Query

RTK Query is Redux Toolkit's opinionated server-state layer.

Use it when:

- your app already uses Redux Toolkit,
- you want generated hooks from API endpoints,
- you want cache invalidation integrated with Redux.

7.1 API Slice Setup

```
// src/services/api.js  
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";  
  
export const api = createApi({  
  reducerPath: "api",  
  baseQuery: fetchBaseQuery({ baseUrl: "/api" }),  
  tagTypes: ["Post"],  
  endpoints: (builder) => ({  
    getPosts: builder.query({  
      query: () => "/posts",  
      providesTags: ["Post"]  
    }),  
    createPost: builder.mutation({  
      query: (newPost) => ({  
        url: "/posts",  
        method: "POST",  
        body: newPost  
      }),  
      invalidatesTags: ["Post"]  
    })  
  })  
});  
  
export const { useGetPostsQuery, useCreatePostMutation } = api;
```

7.2 Add It to the Store

```
import { configureStore } from "@reduxjs/toolkit";
import { api } from "../services/api";

export const store = configureStore({
  reducer: {
    [api.reducerPath]: api.reducer
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(api.middleware)
});
```

7.3 Using RTK Query in a Component

```
import { useGetPostsQuery, useCreatePostMutation } from "../services/api";

function PostsPage() {
  const { data: posts = [], isLoading, isError } = useGetPostsQuery();
  const [createPost, { isLoading: isCreating }] = useCreatePostMutation();

  if (isLoading) return <p>Loading...</p>;
  if (isError) return <p>Could not load posts</p>;

  return (
    <div>
      <button
        disabled={isCreating}
        onClick={() => createPost({ title: "New post" })}
      >
        Add post
      </button>

      <ul>
        {posts.map((post) => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

7.4 Why RTK Query Is Better Than Hand-Rolled Redux Fetch State

It gives you:

- generated hooks,
- cached responses,
- re-fetch triggers,
- invalidation via tags,
- mutation helpers,
- less boilerplate.

Interview line:

- If I already have Redux Toolkit and my problem is API caching rather than arbitrary client workflow, RTK Query is usually the first thing I reach for.

8) TanStack Query / React Query

TanStack Query is for server state.

It is not a replacement for all client state. It is specifically excellent at:

- caching,
- staleness control,
- retry logic,
- background revalidation,
- optimistic mutations,
- pagination and infinite queries.

8.1 Setup

```
npm install @tanstack/react-query
```

8.2 Query Client Setup

```
// src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import App from "./App";

const queryClient = new QueryClient();

ReactDOM.createRoot(document.getElementById("root")).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

8.3 Basic Query Example

```
import { useQuery } from "@tanstack/react-query";

async function fetchUsers() {
  const response = await fetch("/api/users");
  if (!response.ok) {
    throw new Error("Could not fetch users");
  }
  return response.json();
}
```

```

function UsersPage() {
  const { data = [], isLoading, isError, error } = useQuery({
    queryKey: ["users"],
    queryFn: fetchUsers,
    staleTime: 30_000
  });

  if (isLoading) return <p>Loading...</p>;
  if (isError) return <p>{error.message}</p>;

  return (
    <ul>
      {data.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

```

8.4 Mutation Example with Invalidation

```

import { useMutation, useQueryClient } from "@tanstack/react-query";

function AddUserButton() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: async (newUser) => {
      const response = await fetch("/api/users", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(newUser)
      });
    },

    if (!response.ok) throw new Error("Failed to create user");
    return response.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["users"] });
  }
  });

  return (
    <button onClick={() => mutation.mutate({ name: "Taylor" })}>
      Add user
    </button>
  );
}

```

8.5 Optimistic Update Example

```

const mutation = useMutation({
  mutationFn: async (newTodo) => {
    const response = await fetch("/api/todos", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(newTodo)
    });
    return response.json();
  },
  onMutate: async (newTodo) => {
    await queryClient.cancelQueries({ queryKey: ["todos"] });
    const previousTodos = queryClient.getQueryData(["todos"]);

    queryClient.setQueryData(["todos"], (old = []) => [...old, { ...newTodo, id: "temp" }]);

    return { previousTodos };
  },
  onError: (_error, _newTodo, context) => {
    queryClient.setQueryData(["todos"], context.previousTodos);
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ["todos"] });
  }
});

```

8.6 RTK Query vs TanStack Query

Use RTK Query when:

- your app already uses Redux Toolkit heavily,
- you want one integrated state stack,
- team conventions prefer Redux-centric data flow.

Use TanStack Query when:

- you do not want Redux for general state,
- your main complexity is API lifecycle,
- you want a framework-agnostic cache layer.

9) Recoil

Recoil models state as atoms and selectors.

9.1 Atom Example

```

import { atom, RecoilRoot, useRecoilState } from "recoil";

const cartCountState = atom({
  key: "cartCountState",
  default: 0
});

```

```
function CartButton() {
  const [count, setCount] = useRecoilState(cartCountState);

  return <button onClick={() => setCount((current) => current + 1)}>Cart: {count}</button>;
}

function App() {
  return (
    <RecoilRoot>
      <CartButton />
    </RecoilRoot>
  );
}
```

9.2 Selector Example

```
import { selector, useRecoilValue } from "recoil";

const cartTotalState = selector({
  key: "cartTotalState",
  get: ({ get }) => {
    const items = get(cartItemsState);
    return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
  }
});

function CartTotal() {
  const total = useRecoilValue(cartTotalState);
  return <p>Total: ${total}</p>;
}
```

Use Recoil when:

- you want graph-like fine-grained subscriptions,
- state is shared but not naturally centralized in one reducer,
- your team prefers atom/selector modeling.

10) Common Architecture Combinations

10.1 Lean Modern Stack

- local state with React
- form state with `react-hook-form`
- server state with TanStack Query
- URL state in the router
- no Redux unless complexity justifies it

10.2 Enterprise Stack

- Redux Toolkit for shared client workflows
- RTK Query for API data
- route state in the router

- design system for shared UI primitives

10.3 Hybrid Stack

- Redux Toolkit for complex UI/app workflow
- TanStack Query for server state
- URL state for filters/search

This is valid as long as boundaries are explicit.

11) Next.js Deep Dive

Next.js is a React meta-framework optimized for full-stack web applications.

Key concepts to know:

- App Router
- Server Components
- Client Components
- layouts
- route handlers
- streaming
- caching and revalidation
- SSG/SSR/ISR
- metadata APIs

11.1 Server Component Example

```
// app/products/page.jsx
export default async function ProductsPage() {
  const response = await fetch("https://example.com/api/products", {
    next: { revalidate: 60 }
  });

  const products = await response.json();

  return (
    <ul>
      {products.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  );
}
```

Why this is good:

- data fetch stays on server,
- bundle stays smaller,
- revalidation can be configured at fetch boundary.

11.2 Client Component Example

```

"use client";

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount((c) => c + 1)}>{count}</button>;
}

```

Use a Client Component when:

- you need hooks like `useState` ,
- you need browser APIs,
- you need direct user interaction.

11.3 Nested Layout Example

```

// app/dashboard/layout.jsx
export default function DashboardLayout({ children }) {
  return (
    <div>
      <aside>Sidebar</aside>
      <main>{children}</main>
    </div>
  );
}

```

11.4 Route Handler Example

```

// app/api/users/route.js
export async function GET() {
  const users = await db.user.findMany();
  return Response.json(users);
}

export async function POST(request) {
  const body = await request.json();
  const user = await db.user.create({ data: body });
  return Response.json(user, { status: 201 });
}

```

11.5 Metadata Example

```

// app/blog/[slug]/page.js
export async function generateMetadata({ params }) {
  const post = await getPost(params.slug);

  return {
    title: post.title,

```

```

    description: post.summary,
    openGraph: {
      title: post.title,
      description: post.summary
    }
  };
}

```

11.6 SSR vs SSG vs ISR Example

SSG mental model:

```

// static-like page, fully cacheable
export default function AboutPage() {
  return <h1>About us</h1>;
}

```

ISR-like fetch usage:

```

await fetch("https://example.com/api/news", {
  next: { revalidate: 300 }
});

```

Dynamic per-request rendering:

```

await fetch("https://example.com/api/me", {
  cache: "no-store"
});

```

Interview line:

- In Next.js I choose rendering mode per route or even per fetch boundary based on freshness, personalization, SEO, and traffic patterns.

11.7 Suspense and Streaming Example

```

import { Suspense } from "react";
import ProductReviews from "../ProductReviews";

export default function ProductPage() {
  return (
    <div>
      <h1>Product Page</h1>
      <Suspense fallback={<p>Loading reviews...</p>}>
        <ProductReviews />
      </Suspense>
    </div>
  );
}

```

11.8 When to Use Next.js

Use Next.js when:

- you need SSR/SSG/ISR,
- SEO matters,
- you want an integrated full-stack React framework,
- you want route handlers and strong server/client boundaries,
- you care about image/font/script optimization.

12) Remix Deep Dive

Remix is route-centric and request/response-centric.

Key concepts:

- route modules,
- loaders for reads,
- actions for writes,
- forms as first-class primitives,
- nested routes,
- progressive enhancement.

12.1 Loader Example

```
// app/routes/products.jsx
import { json } from "@remix-run/node";
import { useLoaderData } from "@remix-run/react";

export async function loader() {
  const products = await db.product.findMany();
  return json({ products });
}

export default function ProductsRoute() {
  const { products } = useLoaderData();

  return (
    <ul>
      {products.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  );
}
```

12.2 Action Example

```
// app/routes/products.new.jsx
import { json, redirect } from "@remix-run/node";
import { Form, useActionData } from "@remix-run/react";
```

```

export async function action({ request }) {
  const formData = await request.formData();
  const title = formData.get("title");

  if (!title) {
    return json({ error: "Title is required" }, { status: 400 });
  }

  await db.product.create({ data: { title } });
  return redirect("/products");
}

export default function NewProductRoute() {
  const actionData = useActionData();

  return (
    <Form method="post">
      <input name="title" placeholder="Product title" />
      <button type="submit">Create</button>
      {actionData?.error && <p>{actionData.error}</p>}
    </Form>
  );
}

```

12.3 Why Remix Is Strong

- forms map naturally to HTTP writes,
- route-level data is colocated with route UI,
- progressive enhancement is a first-class mental model,
- nested routes scale well.

12.4 When to Use Remix

Use Remix when:

- your app is route-driven,
- form-heavy UX matters,
- request/response semantics are important,
- you want SSR with strong web-standard mental models.

13) Astro

Astro is built around islands architecture: render most of the page as static HTML and hydrate only small interactive islands.

13.1 Astro Component Example

```

---
import Counter from "../components/Counter.jsx";
---

```

```
<html>
  <body>
    <h1>Marketing page</h1>
    <Counter client:load />
  </body>
</html>
```

The `client:load` directive tells Astro to hydrate that React component on the client.

13.2 Why Astro Matters

Use Astro when:

- your site is mostly content,
- interactivity is sparse,
- you want to minimize client-side JavaScript by default.

13.3 Astro vs Next.js

- Astro is great for content-heavy pages with isolated interactivity.
- Next.js is stronger for app-heavy, authenticated, fully interactive systems.

14) Vite

Vite is a fast dev server and build tool, not a full framework.

14.1 Basic React Setup Shape

```
// vite.config.js
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()]
});
```

14.2 Why Teams Choose Vite

- fast startup,
- fast HMR,
- simple config for most apps,
- excellent local development experience.

14.3 When to Use Vite

Use Vite when:

- you want a custom React stack,
- you do not need a meta-framework by default,
- you are building an SPA or your own SSR layer.

15) webpack

webpack is a bundler centered around a dependency graph, loaders, and plugins.

15.1 Basic Config Example

```
const path = require("path");

module.exports = {
  mode: "production",
  entry: "./src/index.js",
  output: {
    path: path.resolve(__dirname, "dist"),
    filename: "bundle.js"
  },
  module: {
    rules: [
      {
        test: /\.jsx?$/,
        exclude: /node_modules/,
        use: "babel-loader"
      }
    ]
  },
  resolve: {
    extensions: [".js", ".jsx"]
  }
};
```

15.2 What to Explain in Interviews

You should know:

- entry,
- output,
- loaders,
- plugins,
- tree shaking,
- code splitting,
- ESM vs CommonJS implications.

16) Micro Frontends and Module Federation

16.1 Module Federation Host Example

```
// webpack.config.js
const ModuleFederationPlugin = require("webpack").container.ModuleFederationPlugin;

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: "host",
      remotes: {
        catalog: "catalog@http://localhost:3001/remoteEntry.js"
      }
    })
  ]
};
```

```

    },
    shared: {
      react: { singleton: true },
      "react-dom": { singleton: true }
    }
  })
]
};

```

16.2 Remote Example

```

const ModuleFederationPlugin = require("webpack").container.ModuleFederationPlugin;

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: "catalog",
      filename: "remoteEntry.js",
      exposes: {
        "./ProductCard": "./src/ProductCard"
      },
      shared: {
        react: { singleton: true },
        "react-dom": { singleton: true }
      }
    })
  ]
};

```

16.3 Runtime Consumption Example

```

const RemoteProductCard = React.lazy(() => import("catalog/ProductCard"));

function HomePage() {
  return (
    <Suspense fallback=<p>Loading catalog widget...</p>>
      <RemoteProductCard />
    </Suspense>
  );
}

```

16.4 MFE vs Shared Component Library

Shared component library:

- build-time integration,
- better for tokens, buttons, inputs, tables,
- still couples consumers to rebuild/redeploy cycle.

Micro frontend:

- runtime composition,
- independent deployment,
- stronger team autonomy,
- higher operational complexity.

Interview line:

- If the goal is only UI reuse, prefer a component library. If the goal is independent deployment and bounded ownership, consider micro-frontends.

17) Authentication and Authorization

17.1 Authentication vs Authorization

- Authentication: who are you?
- Authorization: what are you allowed to do?

Route guards in the UI are not true authorization. The server must enforce it.

17.2 Cookie-Based Session Example

```
// frontend request
await fetch("/api/me", {
  credentials: "include"
});
```

With session auth:

- browser sends cookie automatically,
- server reads session cookie,
- server resolves current user.

Why teams like this:

- `HttpOnly` cookies protect session tokens from direct JS access,
- browser request flow is simpler,
- SSR integration is often cleaner.

17.3 Bearer Token Example

```
const accessToken = sessionStorage.getItem("accessToken");

await fetch("/api/orders", {
  headers: {
    Authorization: `Bearer ${accessToken}`
  }
});
```

This can work, but senior trade-off discussion must mention:

- XSS risk if JS can read the token,
- refresh token handling,
- rotation and revocation strategy.

17.4 Access + Refresh Token Flow Example

```
async function authorizedFetch(url, options = {}) {
  const accessToken = sessionStorage.getItem("accessToken");

  let response = await fetch(url, {
    ...options,
    headers: {
      ...options.headers,
      Authorization: `Bearer ${accessToken}`
    }
  });

  if (response.status === 401) {
    const refreshResponse = await fetch("/api/refresh", { method: "POST", credentials:
"include" });
    if (!refreshResponse.ok) throw new Error("Session expired");

    const { accessToken: nextAccessToken } = await refreshResponse.json();
    sessionStorage.setItem("accessToken", nextAccessToken);

    response = await fetch(url, {
      ...options,
      headers: {
        ...options.headers,
        Authorization: `Bearer ${nextAccessToken}`
      }
    });
  }

  return response;
}
```

17.5 OAuth 2.0 / OIDC Interview Answer

Good practical answer:

- OAuth is for delegated authorization.
- OIDC adds identity.
- For browser apps, auth code flow with PKCE is the modern standard.
- Implicit flow is not the current recommendation.

17.6 BFF Pattern Example

Browser -> BFF -> internal services

Why BFF helps:

- browser avoids holding internal service tokens directly,
- auth/session handling becomes centralized,

- aggregation and transformation stay server-side.

18) Browser Web APIs That Show Up in Grilling Interviews

18.1 fetch + AbortController

```
const controller = new AbortController();

fetch("/api/search?q=react", { signal: controller.signal })
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => {
    if (error.name === "AbortError") {
      console.log("Request cancelled");
    }
  });

controller.abort();
```

Use this for:

- live search,
- route transitions,
- stale request cancellation.

18.2 IndexedDB Example

```
const request = indexedDB.open("notes-db", 1);

request.onupgradeneeded = function () {
  const db = request.result;
  db.createObjectStore("notes", { keyPath: "id" });
};

request.onsuccess = function () {
  const db = request.result;
  const tx = db.transaction("notes", "readwrite");
  tx.objectStore("notes").put({ id: "1", title: "Offline note" });
};
```

Use IndexedDB when:

- data is large,
- you need structured client-side persistence,
- offline support matters.

18.3 Cache API Example

```
async function cacheAvatar() {
  const cache = await caches.open("avatars-v1");
```

```
    await cache.add("/images/avatar.png");  
  }
```

Use Cache API for:

- request/response style caching,
- service-worker-driven offline strategies,
- asset caching.

18.4 postMessage Example

```
window.addEventListener("message", (event) => {  
  if (event.origin !== "https://trusted.example.com") return;  
  console.log("Received", event.data);  
});  
  
iframe.contentWindow.postMessage({ type: "authComplete" }, "https://trusted.example.com");
```

Always validate `origin` .

18.5 BroadcastChannel Example

```
const channel = new BroadcastChannel("auth-events");  
  
channel.postMessage({ type: "logout" });  
  
channel.onmessage = (event) => {  
  if (event.data.type === "logout") {  
    console.log("Log out this tab too");  
  }  
};
```

Useful for:

- logout sync across tabs,
- simple same-origin coordination.

18.6 IntersectionObserver Example

```
const observer = new IntersectionObserver((entries) => {  
  entries.forEach((entry) => {  
    if (entry.isIntersecting) {  
      console.log("Element became visible");  
    }  
  });  
});  
  
observer.observe(document.getElementById("lazy-image"));
```

Use for:

- lazy loading,
- infinite scroll triggers,
- view-based analytics.

18.7 Web Worker Example

```
const worker = new Worker(new URL("./worker.js", import.meta.url));
worker.postMessage({ numbers: [1, 2, 3, 4] });
worker.onmessage = (event) => {
  console.log("Worker result", event.data);
};
```

worker.js :

```
self.onmessage = (event) => {
  const sum = event.data.numbers.reduce((a, b) => a + b, 0);
  self.postMessage(sum);
};
```

Use workers for CPU-heavy work that would otherwise block the main thread.

19) PWA, Service Workers, Offline, and Push Notifications

19.1 Service Worker Registration

```
if ("serviceWorker" in navigator) {
  navigator.serviceWorker.register("/service-worker.js");
}
```

19.2 Basic Service Worker Caching Example

```
self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open("app-shell-v1").then((cache) => {
      return cache.addAll(["/", "/styles.css", "/app.js"]);
    })
  );
});

self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((cachedResponse) => {
      return cachedResponse || fetch(event.request);
    })
  );
});
```

19.3 Push Notification Flow Example

Client subscription:

```
const registration = await navigator.serviceWorker.ready;
const subscription = await registration.pushManager.subscribe({
  userVisibleOnly: true,
  applicationServerKey: "PUBLIC_VAPID_KEY"
});

await fetch("/api/push-subscriptions", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(subscription)
});
```

Service worker push handler:

```
self.addEventListener("push", (event) => {
  const data = event.data?.json() ?? { title: "New notification" };

  event.waitUntil(
    self.registration.showNotification(data.title, {
      body: data.body,
      icon: "/icon.png"
    })
  );
});
```

Important interview answer:

- Notifications can appear even when the page is closed because the browser can wake the service worker when a push event arrives.

20) Routing Best Practices

20.1 Route State in the URL

```
import { Link } from "react-router-dom";

function UserList({ users }) {
  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>
          <Link to={` /users/${user.id}`}>{user.name}</Link>
        </li>
      ))}
    </ul>
  );
}
```

Best practices:

- code split at route boundaries,
- colocate data with route where framework supports it,
- use URL for deep-linkable state,
- define loading and error boundaries explicitly.

21) Testing Strategy

21.1 Component Test Example

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import Counter from "../Counter";

test("increments count when button is clicked", async () => {
  const user = userEvent.setup();
  render(<Counter />);

  await user.click(screen.getByRole("button", { name: /increment/i }));
  expect(screen.getByText("1")).toBeInTheDocument();
});
```

21.2 Reducer Unit Test Example

```
import cartReducer, { itemAdded } from "../cartSlice";

test("adds a new item to cart", () => {
  const state = { items: [], coupon: null };
  const nextState = cartReducer(state, itemAdded({ id: "1", name: "Bag", price: 20 }));

  expect(nextState.items).toHaveLength(1);
  expect(nextState.items[0].quantity).toBe(1);
});
```

21.3 E2E Flow Example

```
// Playwright style example
await page.goto("/checkout");
await page.getByRole("button", { name: "Add shoe" }).click();
await page.getByRole("button", { name: "Checkout" }).click();
await expect(page.getByText("Order confirmed")).toBeVisible();
```

What to test:

- reducer logic at unit level,
- components by user behavior,
- critical journeys with E2E,
- avoid testing implementation details.

22) Design Patterns and Architecture Talking Points

22.1 Container vs Presentational Components

Conceptual split:

- container: fetches data, coordinates state,
- presentational: renders UI from props.

Example:

```
function UserListContainer() {
  const { data = [] } = useQuery({ queryKey: ["users"], queryFn: fetchUsers });
  return <UserListView users={data} />;
}

function UserListView({ users }) {
  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

22.2 Custom Hooks

```
function useCurrentUser() {
  return useQuery({
    queryKey: ["current-user"],
    queryFn: async () => {
      const response = await fetch("/api/me");
      return response.json();
    }
  });
}
```

22.3 HOC vs Render Props vs Hooks

Interview answer:

- HOCs are useful for wrapper-style reuse but can produce tree nesting and prop collisions.
- Render props make reuse explicit but can be verbose.
- Hooks became the preferred reuse primitive for stateful logic.

23) Performance, SEO, and Bundle Strategy

23.1 Lazy Loading Example

```
const ProductChart = React.lazy(() => import("./ProductChart"));

function Dashboard() {
  return (
    <Suspense fallback={<p>Loading chart...</p>}>
      <ProductChart />
    </Suspense>
  );
}
```

23.2 React.memo Example

```
const ProductRow = React.memo(function ProductRow({ product, onSelect }) {
  return <li onClick={() => onSelect(product.id)}>{product.name}</li>;
});
```

23.3 SEO Example in Server Frameworks

Strong SEO answer includes:

- server-rendered or static HTML where appropriate,
- metadata correctness,
- canonical URLs,
- structured data when relevant,
- fast page loads,
- meaningful internal linking.

Interview line:

- SEO is not just SSR. It is discoverability, metadata, crawlability, performance, and correct content exposure.

24) Final Interview Answers

24.1 "How would you choose state management for a new React app?"

Strong answer:

- I first classify the state: local UI state, URL state, server state, and shared client workflow state. I keep local UI state in React, use the URL for shareable navigational state, use TanStack Query or RTK Query for server-state caching, and only add Redux Toolkit if the app has enough shared client-side workflow complexity to justify centralized event-driven state.

24.2 "Redux Toolkit or TanStack Query?"

Strong answer:

- They solve different problems. Redux Toolkit is for shared client-state workflows and explicit transitions. TanStack Query is for server-state lifecycle. In a large app I may use both, but if my main complexity is data fetching and caching, I would reach for a query library first.

24.3 "When would you choose Next.js over Vite?"

Strong answer:

- I choose Next.js when I need SSR, SSG, ISR, route handlers, metadata, image optimization, and strong server/client rendering boundaries. I choose Vite when I want a lighter custom React stack and do not need a full meta-framework.

24.4 "When would you choose micro-frontends?"

Strong answer:

- Only when independent deployment and bounded team ownership are real requirements. If the main need is shared UI consistency, I would use a shared component library instead.

25) Senior Revision Checklist

- ☐ Explain the different kinds of state before naming a library.
- ☐ Explain store setup, provider wiring, slice creation, selectors, and dispatch flow in Redux Toolkit.
- ☐ Explain what thunk is, why middleware is needed, and when to prefer `createAsyncThunk`.
- ☐ Explain RTK Query and TanStack Query with real examples.
- ☐ Explain Recoil as an atom-selector model with realistic trade-offs.
- ☐ Explain SSR, SSG, ISR, and streaming with Next.js examples.
- ☐ Explain Remix loaders/actions with form examples.
- ☐ Explain Astro islands and when they outperform heavier app frameworks.
- ☐ Explain Vite vs webpack clearly.
- ☐ Explain Module Federation and micro-frontends with config-level examples.
- ☐ Explain cookies vs localStorage vs memory for auth.
- ☐ Explain CSRF vs XSS and why server-side enforcement still matters.
- ☐ Explain IndexedDB, Cache API, workers, observers, and push notifications with concrete use cases.
- ☐ Explain testing strategy across reducers, components, and E2E.
- ☐ Explain performance and SEO as practical engineering decisions, not slogans.